



Architekturen visualisieren und Fremdsysteme anbinden

Programm für heute



QA|WARE

Vorlesung

1. Ergebnisvorstellung arc42
2. Einführung in das C4 Modell
3. Softwarearchitektur visualisieren
4. Exkurs: Fremdsysteme anbinden
5. Vorstellung von ArchUnit als Testframework für Softwarearchitektur

Übung

1. Visualisieren einer textuellen Softwarearchitektur
2. Einstiegs ins Testen von Softwarearchitektur mit ArchUnit

Wie visualisiert ihr Softwarearchitektur?



QA|WARE

Wie visualisiert ihr Softwarearchitektur?
Welche Tools kennt ihr bereits?



QA|WARE

Ergebnisvorstellung arc42

Wir bauen eine E-Learning Plattform.



QA|WARE

Ihr seid das Architekturteam für ein neues Produkt: eine **moderne E-Learning-Plattform**. Ziel ist es, Dozent:innen und Lernenden eine benutzerfreundliche, skalierbare und sichere Umgebung bereitzustellen, um Kurse online anzubieten und zu absolvieren.

Folgende Kernfunktionen sind angedacht:

- Nutzerregistrierung und Rollenmanagement (Lernende, Dozierende, Admins)
- Kursverwaltung (Kurse erstellen, bearbeiten, veröffentlichen)
- Materialien hochladen (Videos, PDFs, Aufgaben)
- Prüfungen und Quiz-Funktionalität
- Lernfortschritt und Bewertungen
- Optionale Live-Sessions (z. B. mit externer Videoplattform integriert)
- Mobile App und Web-Zugang

Wir bauen eine E-Learning Plattform.



QA|WARE

Gruppe	arc42 Abschnitt	Aufgabe
Gruppe 1	1. Einleitung und Ziele	Beschreibt die Vision der Plattform, benennt zentrale Stakeholder und formuliert deren Erwartungen. Definiert die wichtigsten Qualitätsziele (z. B. Usability, Skalierbarkeit, Wartbarkeit).
Gruppe 2	2. Randbedingungen	Sammelt und beschreibt funktionale und nicht-funktionale Anforderungen, die aus architektonischer Sicht relevant sind (z. B. Datenschutz, Nutzerzahlen, Gerätevielfalt).

Wir bauen eine E-Learning Plattform.



QA|WARE

Gruppe	arc42 Abschnitt	Aufgabe
Gruppe 3	3. Kontextabgrenzung	Beschreibt externe Systeme (z. B. Payment-Anbieter, Videostreaming, Authentifizierung). Beschreibt Schnittstellen und Kommunikationswege.
Gruppe 4	4. Lösungsstrategie	Beschreibt die groben architektonischen Grundideen, z. B. Verwendung von Microservices, Trennung von Frontend/Backend, Cloud-Hosting oder Open-Source-Komponenten.

Wir bauen eine E-Learning Plattform.



QA|WARE

Gruppe	arc42 Abschnitt	Aufgabe
Gruppe 5	6. Laufzeitsicht	Beschreibt einen typischen Ablauf mit Fokus auf Interaktion zwischen Komponenten, z. B. „Lernender meldet sich zu einem Kurs an und startet ein Video“. Nutzt z. B. Sequenzdiagramme.
Gruppe 6	7. Verteilungssicht	Erstellt eine Übersicht, wie das System in einer Produktivumgebung betrieben werden kann (z. B. Web-Frontend, REST-API, Datenbank, Load Balancer, Docker, Cloud).

Wir bauen eine E-Learning Plattform.



QA|WARE

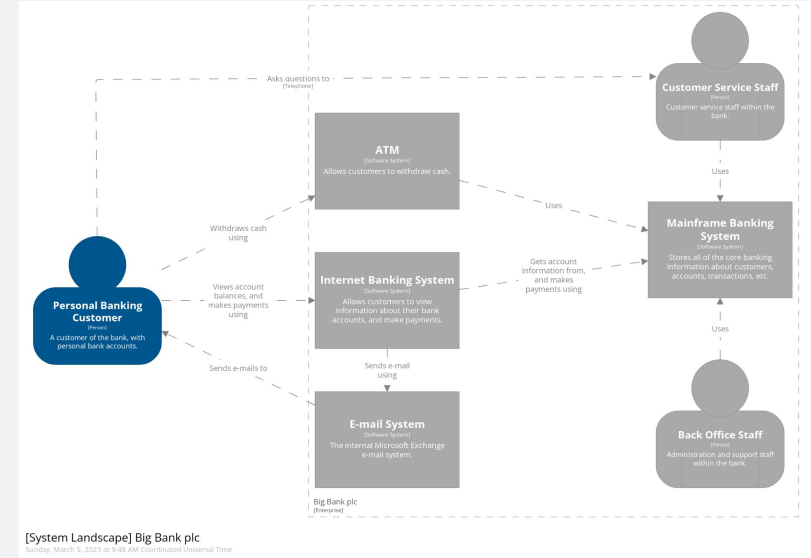
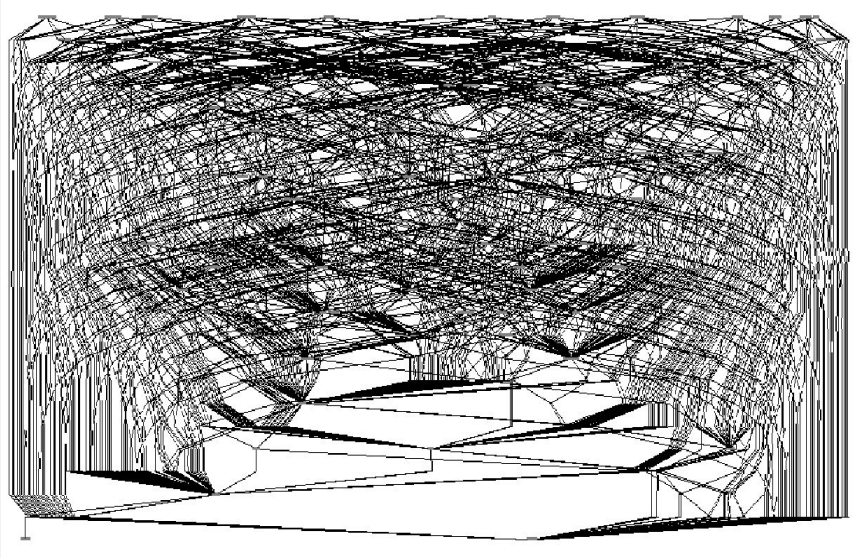
Gruppe	arc42 Abschnitt	Aufgabe
Gruppe 7	10. Qualitätsanforderungen	Erstellt konkrete Qualitätsszenarien zu Anforderungen wie Performanz, Verfügbarkeit, Erweiterbarkeit, Sicherheit usw. (z. B. mit ATAM oder Szenario-Tabellen).
Gruppe 8	11. Risiken und technische Schulden	Identifiziert zentrale technische Risiken (z. B. Datenschutz, Komplexität, Third-Party-Abhängigkeiten) und überlegt, wie diese reduziert oder dokumentiert werden können.



QA|WARE

Das C4 Modell

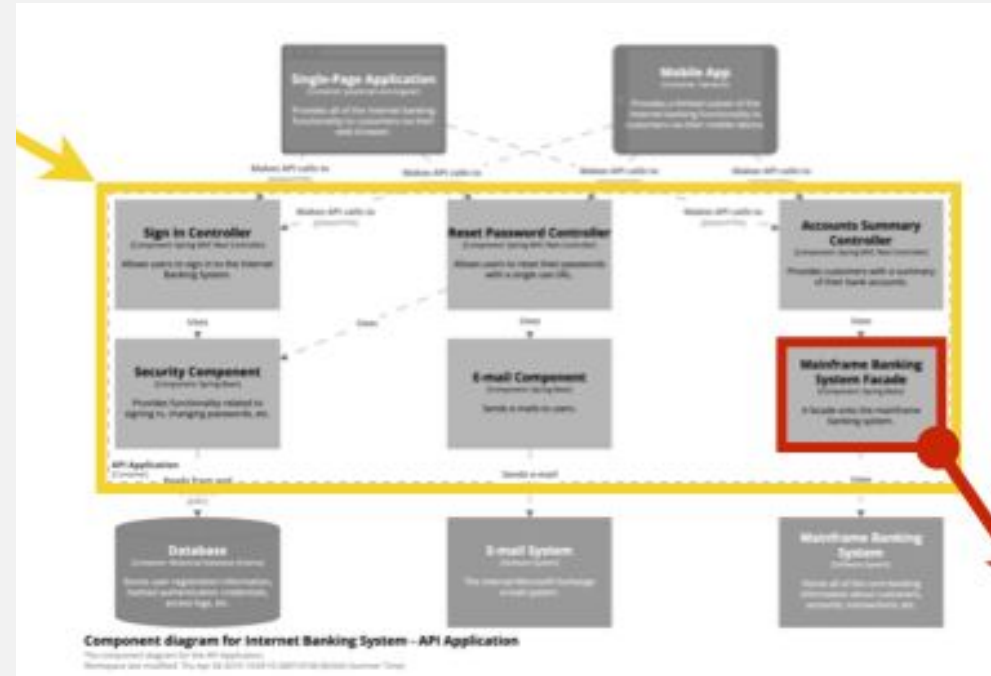
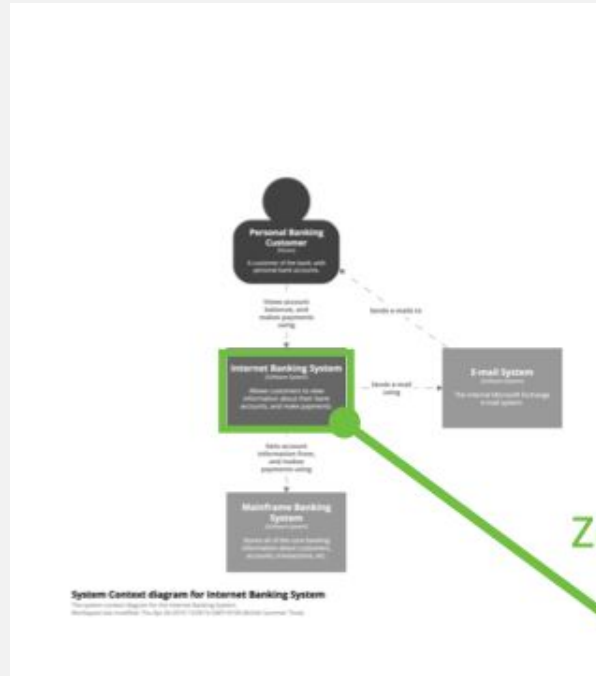
Welches Architekturbild ist besser?



Welches Architekturbild ist besser?



QA|WARE



Ein gutes Architekturbild liegt im Auge des Betrachters.



QA|WARE

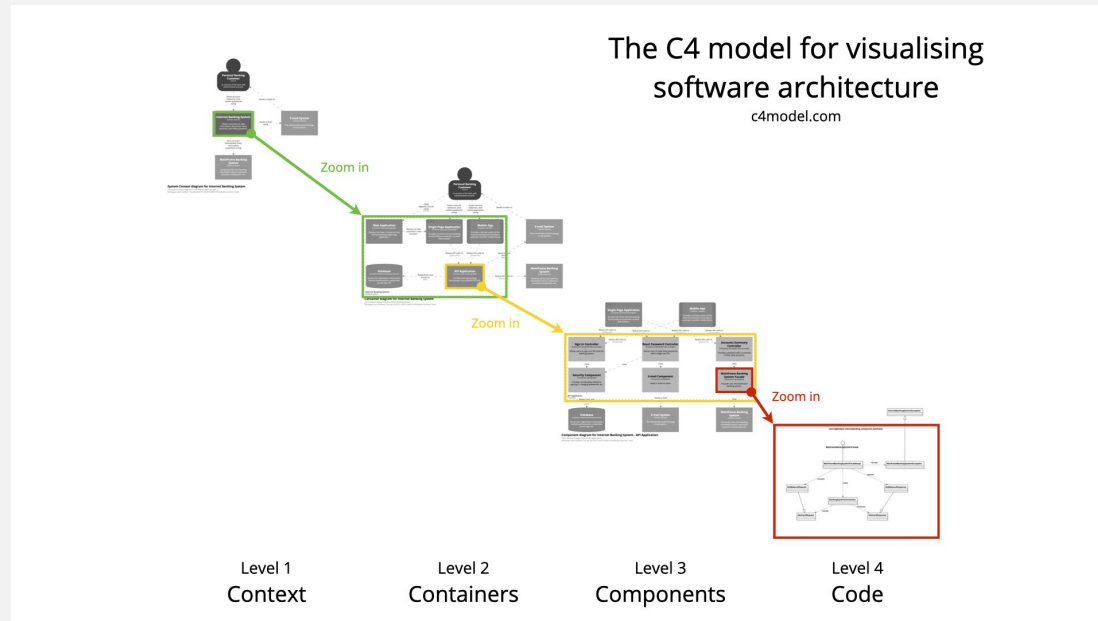
- Je nach Betrachter:in brauchen wir unterschiedliche Flughöhen über unsere Architektur
- Ein CEO muss nicht wissen, wie sich bpsw. die Payment Komponente zusammensetzt und ob Daten lokal oder in der Cloud gespeichert werden. Er/Sie muss aber sehr wohl wissen, dass es eine Payment Komponente gibt, mit der Kund:innen kommunizieren können
- Eine Entwicklerin, die neu ins Team kommt und ein kleines Feature umsetzen soll, muss nicht wissen welche Services in unserem Kubernetes Cluster laufen, aber möchte wissen welche Klassen es in dem Service gibt, den sie erweitern soll

Das C4 Modell beschreibt Softwarearchitektur auf 4 Ebenen.



QA|WARE

Das C4 Modell löst genau dieses Problem, indem es klare Vorgaben macht, wie Architektur in 4 Abstraktionsebenen beschrieben werden kann.



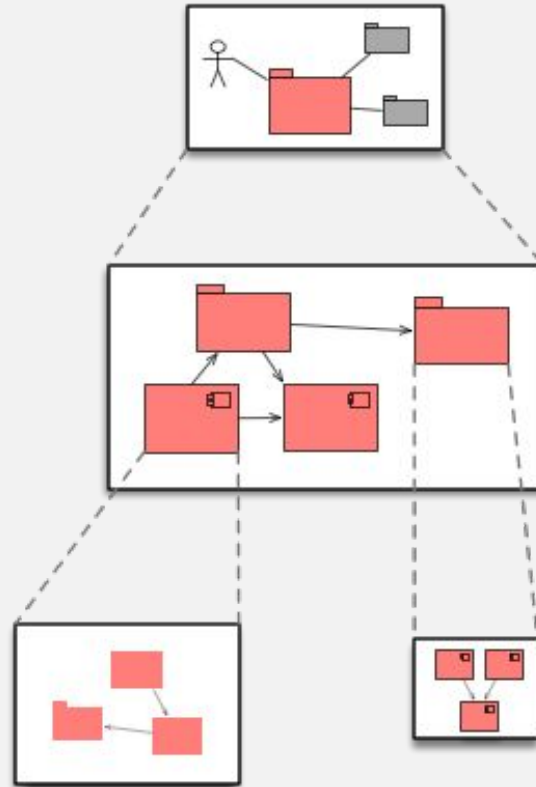
Das C4 Modell beschreibt Softwarearchitektur auf 4 Ebenen.



- Entwickelt vom Softwarearchitekt Simon Brown zwischen 2006 und 2011
- Seit einem 2018 veröffentlichten Artikel und einer entsprechenden Website zunehmend populär (<https://c4model.com/>)
- Ideal geeignet für Abschnitt 5 von arc42 - Bausteinsicht
- Notations- und toolingunabhängig

5. Bausteinsicht

RECAP

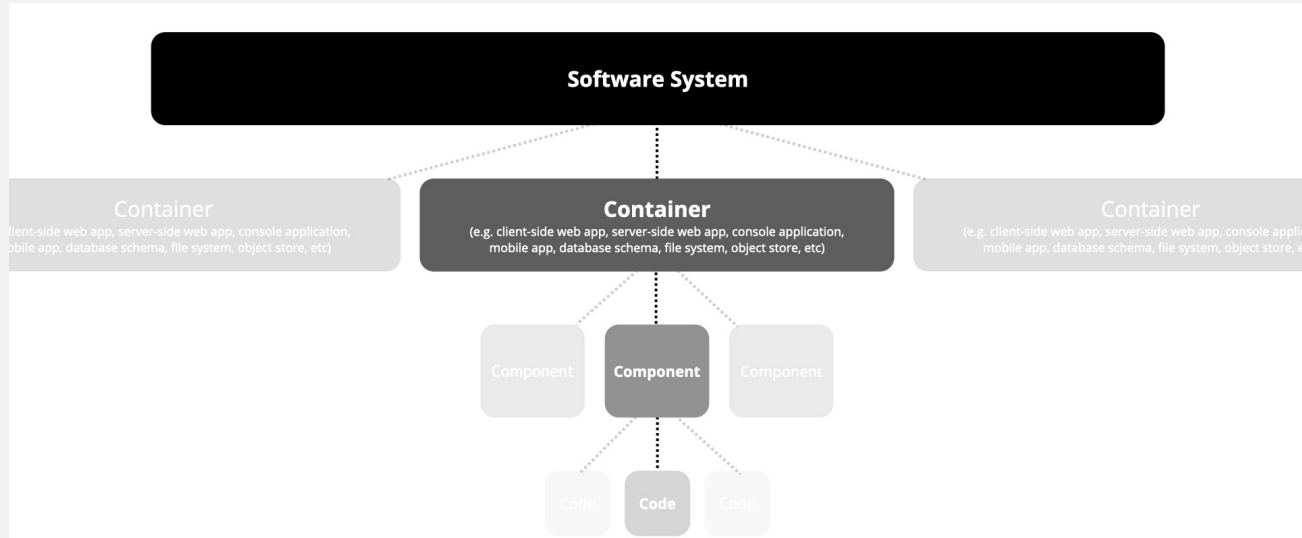


* Hierzu werden wir uns in einer der folgenden Veranstaltungen noch einmal ein weiteres Modell ansehen

Auf oberster Ebene liegen Softwaresystem und User.



QA|WARE

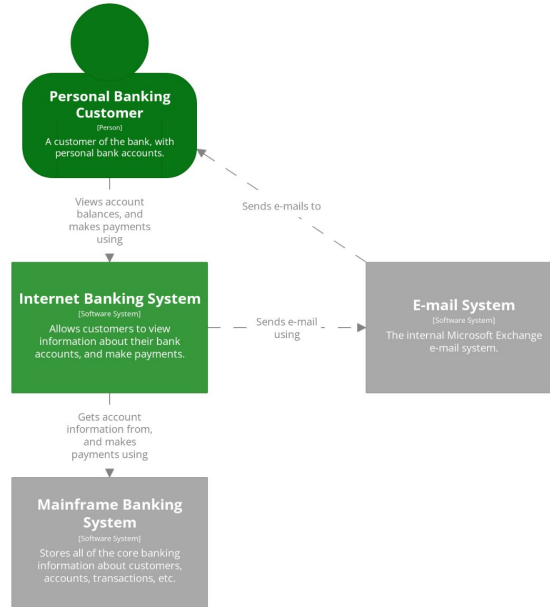


A **software system** is made up of one or more **containers** (applications and data stores), each of which contains one or more **components**, which in turn are implemented by one or more **code** elements (classes, interfaces, objects, functions, etc).

Auf oberster Ebene liegen Softwaresystem und User.



QA|WARE



[System Context] Internet Banking System

The system context diagram for the Internet Banking System - diagram created with Structurizr.
Wednesday, March 22, 2023 at 8:16 AM Coordinated Universal Time

Ein Softwaresystem besteht aus mehreren Containern.



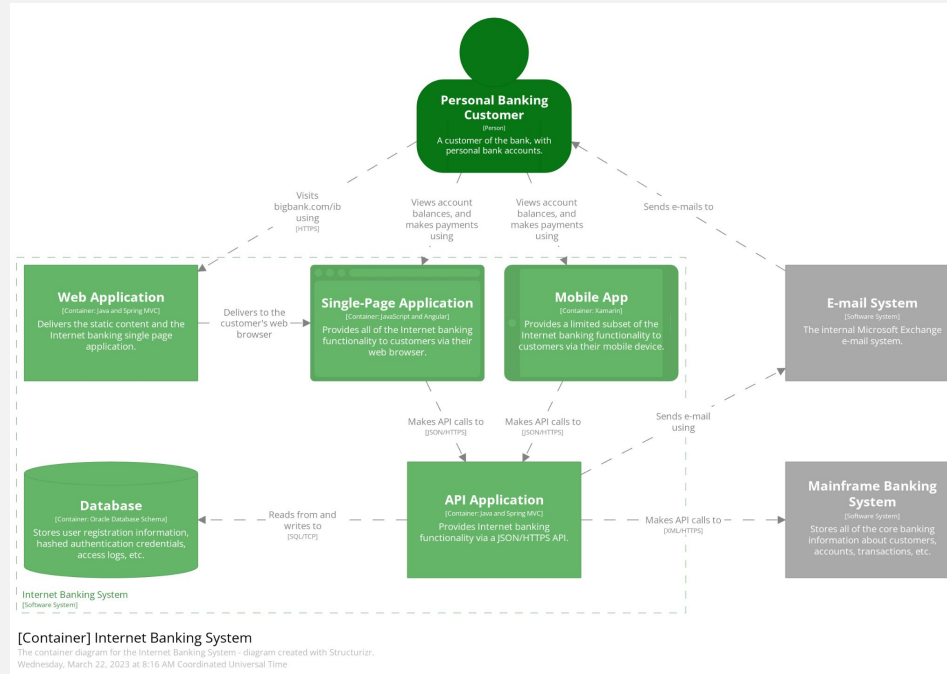
QA|WARE

- Repräsentieren eine Anwendung oder einen Data-Store
- Ein Container ist ein Teil, der laufen muss, damit das Gesamtsystem funktionieren kann
- Beispiele:
 - Server-side web application
 - Client-Side desktop application
 - Datenbank
 - Shell script
 - ...

Ein Softwaresystem besteht aus mehreren Containern.



QAWARE



Ein Container besteht aus mehreren Komponenten.



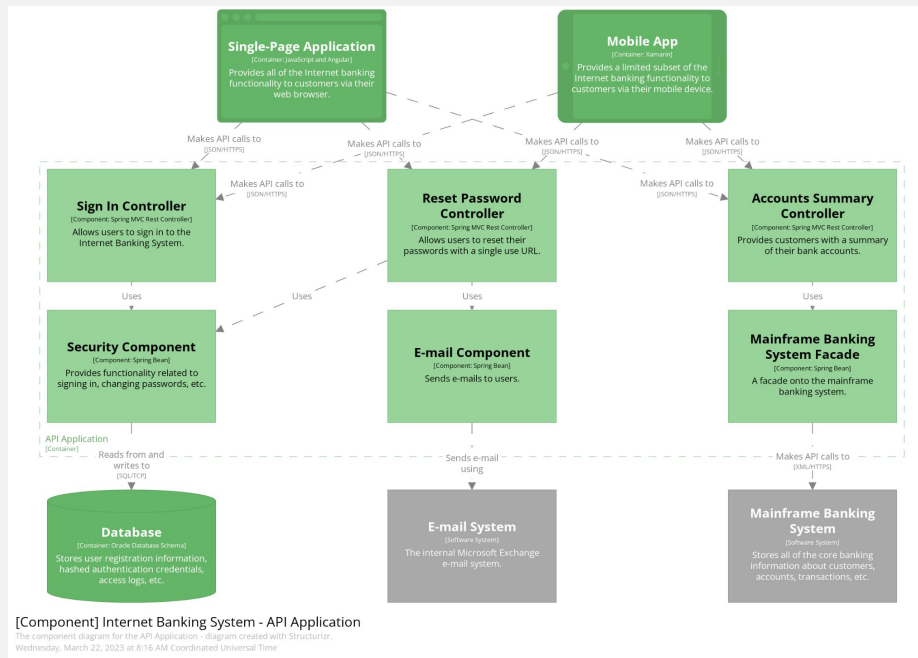
QA|WARE

- im C4 Modell vereint eine Komponente eine Gruppe thematisch zusammenhängender Funktionalitäten hinter einem genau definierten Interface
- In Java, C# etc. eine Sammlung von Klassen und Services hinter einem Interface
- Komponenten im C4 Modell sind nicht selbständig deploybare Einheiten. Der Container ist die deploybare Einheit
- Alle Komponenten innerhalb eines Containers laufen im selben Prozessraum

Ein Container besteht aus mehreren Komponenten.



QA|WARE



Innerhalb einer Komponente lebt der Code.



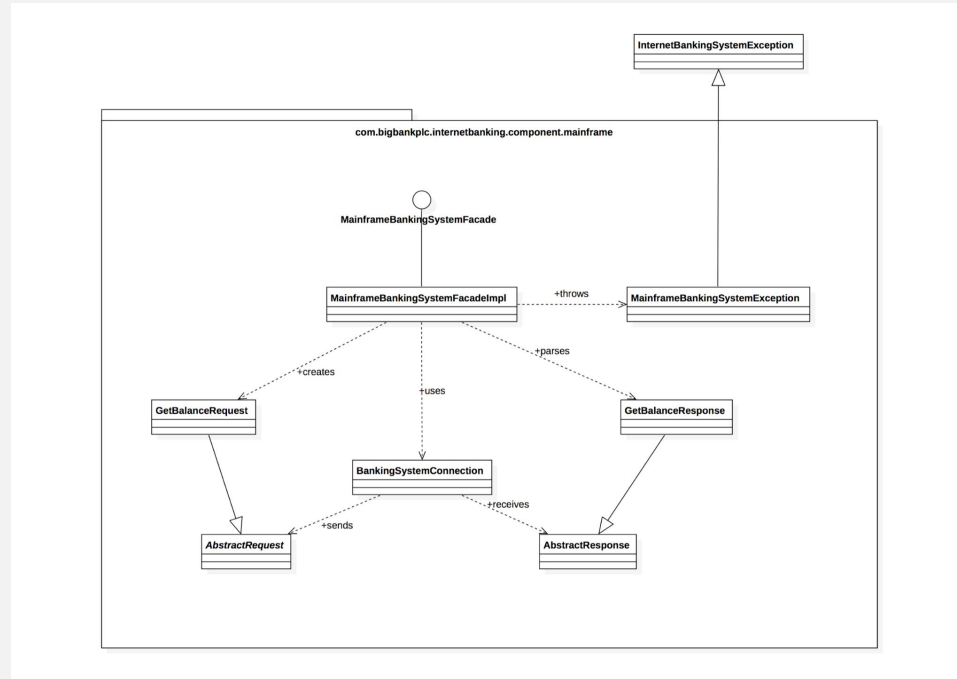
QA|WARE

- Komponenten bestehen aus mehrere Code-Bausteinen
- Ein Code-Baustein ist dann bpsw.:
 - Klasse
 - Interface
 - Enums
 - Funktionen
 - ...

Innerhalb einer Komponente lebt der Code.



QA|WARE



Code oder Komponente?



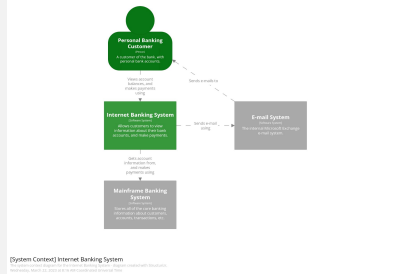
QA|WARE

- Komponenten sind eine Art ein Abstraktionslevel nach oben zu wandern von den Code-Bausteinen, die die jeweilige Programmiersprache bietet.
- Beispiele:
 - **OOP**: Komponente besteht aus Klassen und Interfaces
 - **Procedural programming languages**: Komponente besteht aus mehreren C-Files in einer bestimmten Reihenfolge
 - **JavaScript**: Komponente ist ein JavaScript Modul, das aus einer Anzahl Funktionen und Objekten besteht
 - **Functional programming languages**: Komponente ist ein Modul, das aus mehreren Funktionen, Typen, etc. besteht.

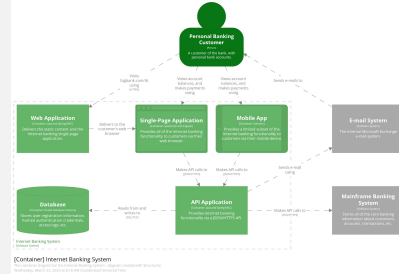
C4 funktioniert wie eine interaktive Landkarte.



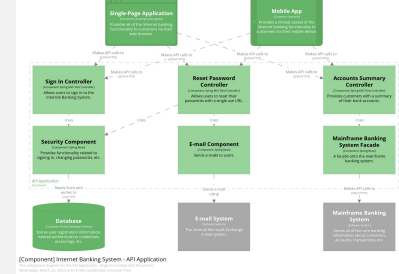
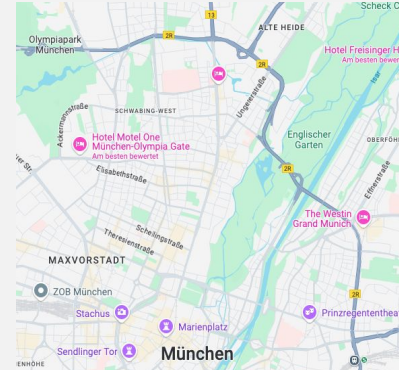
QA|WARE



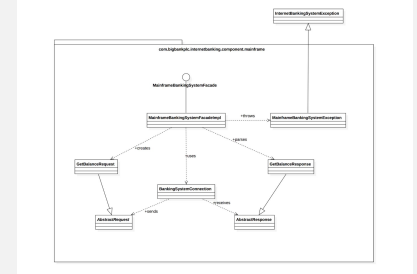
Systemkontextdiagramm
m
QAware



Containerdiagramm



Komponentendiagramm
m



Codediagramm

C4 entwickelt zunehmende Diagrammreife.



QA|WARE

Software architecture diagramming maturity model

Level 1	Level 2	Level 3	Level 4	Level 5
Initial No software architecture diagrams.	Ad hoc Software architecture diagrams with ad hoc abstractions and notation, in a general purpose diagramming tool.	Defined Software architecture diagrams with defined abstractions and notation, in a general purpose diagramming tool.	Modelled Software architecture diagrams with defined abstractions and notation, in a modelling tool, authored manually.	Optimising - Model elements are shared between teams across the organisation. - Models are used as queryable datasets. - Automatic generation of model elements from source code, deployment environment, logs, etc. - etc

..... C4 model



QA|WARE

Exkurs: Fremdsysteme anbinden

Architekturpatterns zur Anbindung von Fremdsystemen.



QA|WARE

Was sollten wir beim Anbinden von
Fremdsystemen beachten? Was kann
schiefgehen?

Wir sollten uns diese Fragen stellen, wenn wir Fremdsysteme anbinden.

- Aus Business-Sicht: Bietet das Fremdsystem alle Funktionen, die ich benötige?
- In welchem Zustand befindet sich das Fremdsystem?
- Weist das Fremdsystem bekannte Sicherheitslücken auf?
- Ist Long-Term Support gewährleistet?
- Gibt es Alternative, die vielleicht besser geeignet sind?

→ Hierbei kann euch der Open-Source Quickcheck helfen

→ Die Entscheidung für einen bestimmten externen Service sollte in einem ADR festgehalten werden



Thema	Checkpunkte	Quellen
Lizenz	- Nur schwaches Copyleft mit kommerzieller Nutzungsmöglichkeit - OSI-konforme Lizenz z. B. LGPL, Apache License, MIT-License - NICHT GPL	ohloh.net, tldrlegal.com opensource.org
Reifegrad	- Stable Release mit Versionsnummer > 1.0 existiert - Projekt existiert länger als ein Jahr	mvnrepository.com
Support	- Community-Bereich und Issue Tracker vorhanden - Durchschnittliche Antwortzeit in der Community < 24 Stunden.	stackoverflow.com
Maven/Gradle	- Aktuelle stabile Version auf mvnrepository.com oder anderen bekannten Repository verfügbar	mvnrepository.com
Dokumentation	- API-Dokumentation - Getting Started Guide/Tutorials - Nutzer-Dokumentation zur aktuellen Version vorhanden.	Projektwebsite
Mängel/Bugs	- Keine offenen kritischen Bugs oder Sicherheitslöcher in der aktuellen Stable-Version	cve.org
Innere Qualität	- Build-Skript vorhanden und baut - Wenig Code-Anomalien/Zyklen. Testüberdeckung > 30%, Comments > 15% - Übersichtliche Struktur/Abhängigkeiten	ohloh.net, FindBugs, Sonar
Projekt-Aktivität	- Projekt ist auf Ohloh gelistet - In den letzten 12 Monaten mind. ein Release - min. 3 aktive Committer - Stetige Commit-Aktivität (mind. 1x pro Monat)	ohloh.net, mvnrepository.com
Bekanntheitsgrad	- Vergleich öffentliches Interesse in Form von Blog-Einträgen, stackoverflow-Fragen oder Google Suchen zu ähnlichen Bausteinen.	Google, Google Trends, Stackoverflow

Unsere Applikation sollte nicht über die Implementierung des Fremdsystems Bescheid wissen.



- Unsere Businesslogik sollte komplett entkoppelt sein von der spezifischen Implementierung des externen Services (Keine Klassen/Objekte etc. direkt in unserem Code verwenden)
- Dafür gibt es einige Architekturpatterns, die unsere Businesslogik von der Implementierung des Fremdsystems trennen
- Alle diese Patterns “verstecken” in unterschiedlicher Tiefe die Implementierung des externen Services

Adapter

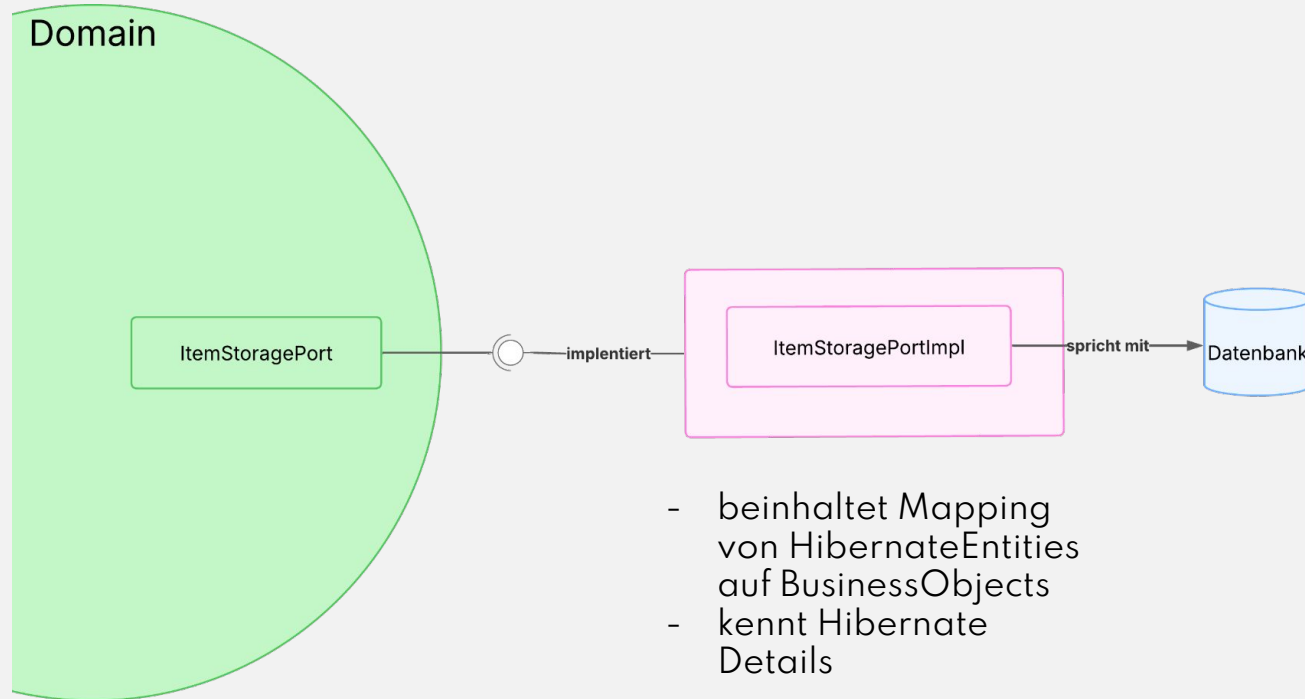
- **Verwandelt ein Interface** in ein anderes Interfaces
- Hexagonale Architektur ist nach diesem System aufgebaut
- Beispiel: Datenbankadapter in Verbindung mit Hibernate oder JOOQ

Adapter

RECAP



QA|WARE



Proxy



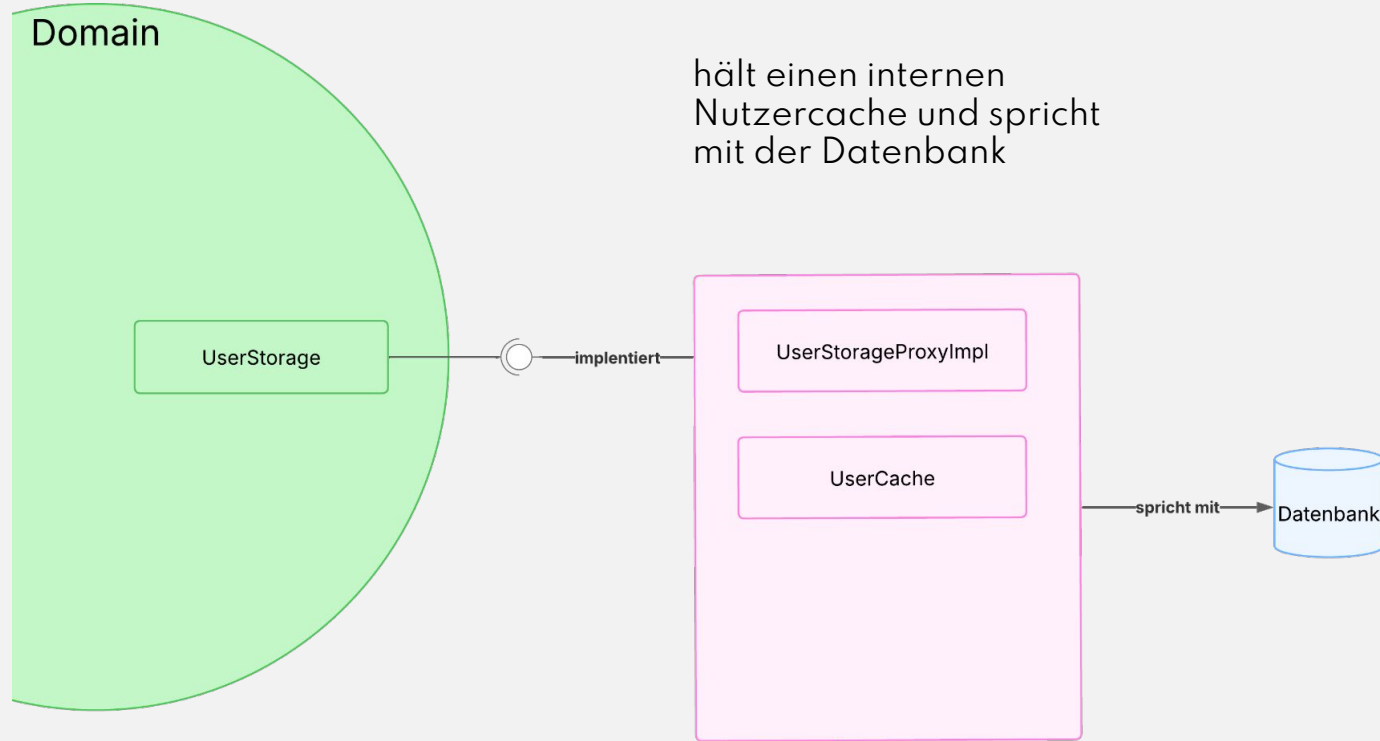
QA|WARE

- **Kapselt Logik**, die vor oder nach dem Aufruf des externen Services notwendig ist
- Dies kann beispielsweise ein interner Cache oder eine weitere Security-Layer sein
- Beispiel: Aufrufen eines externen UserStores mit anschließendem Caching

Proxy



QAWARE



Facade



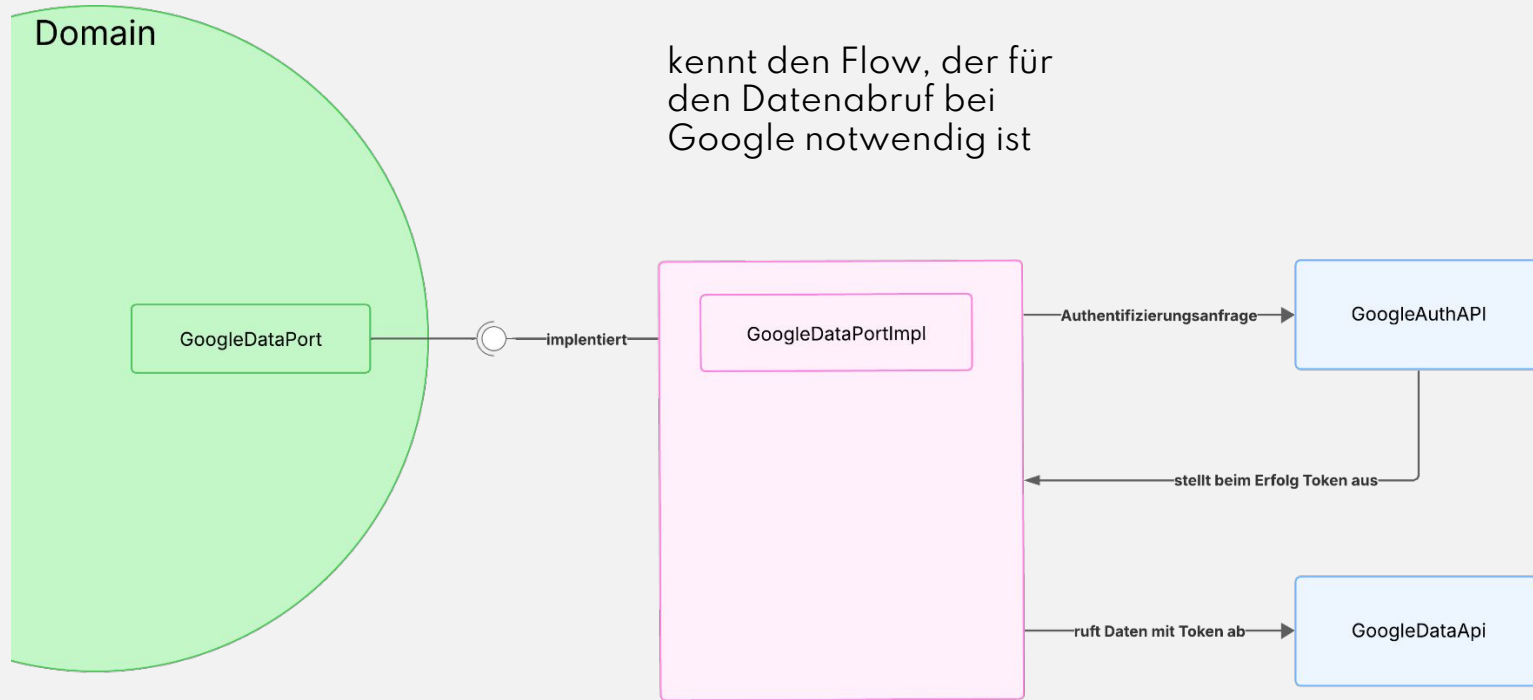
QA|WARE

- Stellt ein gemeinsames Interface für mehrere Interfaces eines externen Services zur Verfügung
- Aufrufen eines externen Services, der aus mehreren Schritten besteht
- Beispiel: Aufrufen eines Google Endpunktes, der Authentifizierung benötigt. Zuerst authentifizieren wir uns bei Google, bevor wir dann die Daten abrufen können

Facade



QA|WARE



Strategy / Policy



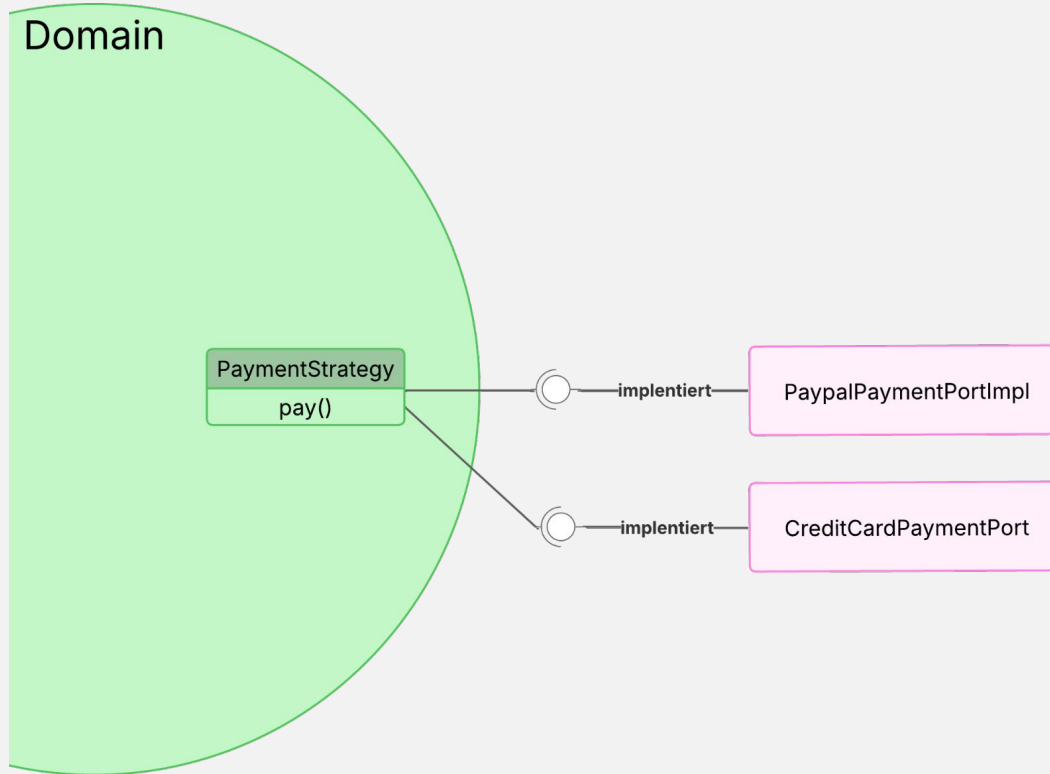
QA|WARE

- Die Strategy stellt ein Interface zur Verfügung, dass einen Algorithmus anwendet
- Für jede mögliche Ausprägung des Algorithmus gibt es dann entsprechende Implementierungen, die beliebig ausgetauscht werden können
- Beispiel: Bezahlung in einer App möglich mit Paypal und Kreditkarte

Strategy / Policy



QA|WARE



Template



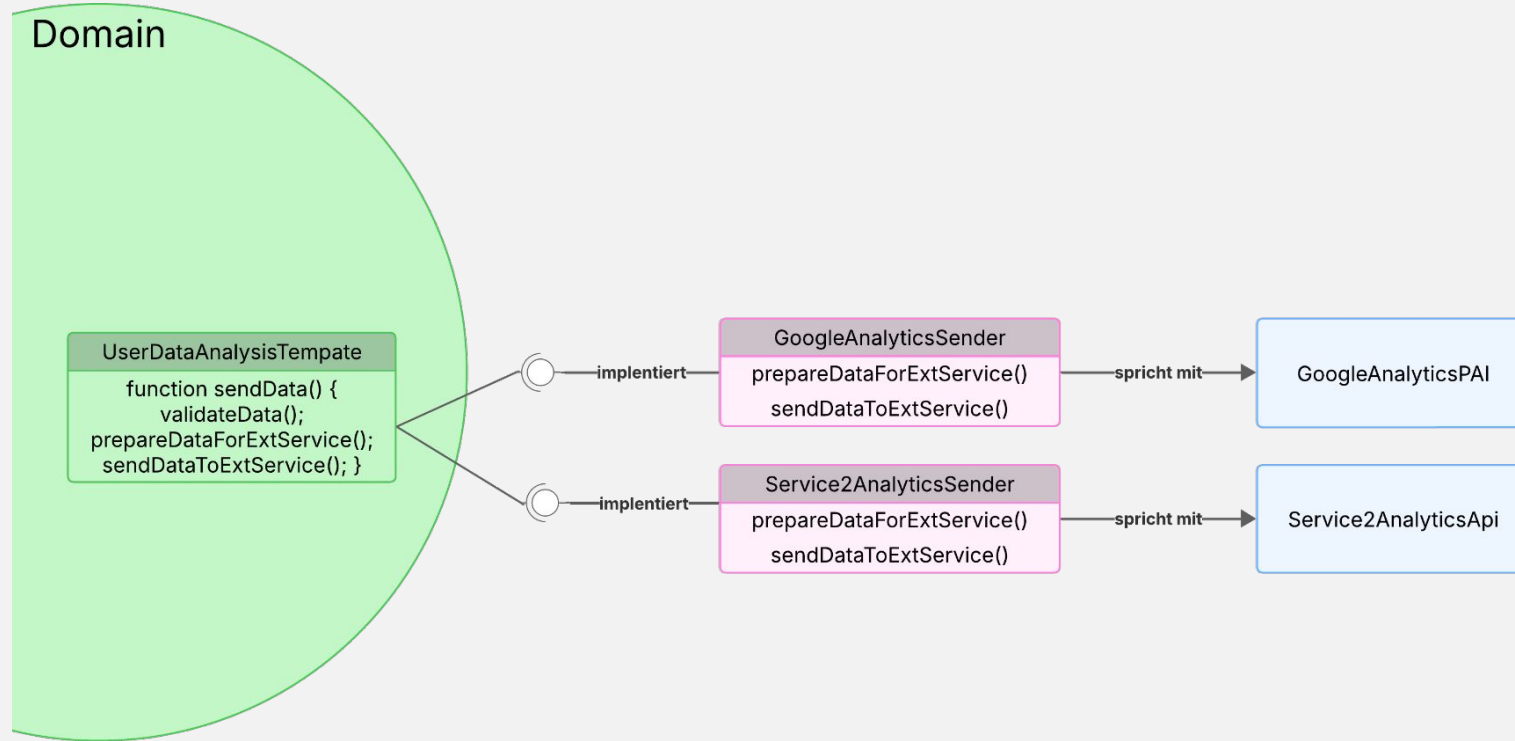
QA|WARE

- Stellt eine Bauanleitung für einen Algorithmus zur Verfügung (Step1 → Step2 → Step3 usw.)
- Implementierungen können dann einzelne Schritte des Algorithmus entsprechend den Anforderungen des externen Services überschreiben
- Beispiel: Schicken von Nutzer:innenanalysedaten an Datenanalysetools

Template



QA|WARE



Es gibt verschiedene Maßnahmen, um Ausfällen von Fremdsystemen entgegenzuwirken.



Architekturpatterns stellen die Grundlage dar, um überhaupt sinnvoll auf Ausfälle von Fremdsystemen reagieren zu können. Ausfälle lassen sich so immer getrennt vom Businesscode der Anwendung behandeln.



Architekturpatterns zur Anbindung von Fremdsystemen.



QA|WARE

Welche Mechanismen kennt ihr, um
Problemen beim Anbinden von
Fremdsystemen vorzubeugen?

Es gibt verschiedene Maßnahmen, um Ausfällen von Fremdsystemen entgegenzuwirken.



QA|WARE

Darüber hinaus existieren weitere Patterns, um mit Ausfällen umzugehen:

- **Retry:** Fehlerhafte Request werden erneut geschickt
- **Circuit Breaker:** Nach einer konfigurierten Anzahl an fehlerhaften Request, wird die Kommunikation mit dem externen Service eingestellt
- **Queuing:** Aufrufe an den externen Service werden asynchron in einer Queue abgearbeitet (Beispiel Mailversand über externen SMTP Server)
- **Caching:** Häufige Aufrufe an einen externen Service werden gecached
- **Fallback:** Für fehlerhafte Aufrufe existiert eine Fallback-Strategie (Caching kann eine solche Strategie sein)
- **Bulkhead:** Begrenzt die Anzahl und die maximale Dauer von Request an externe Services

Für alle Patterns gibt es bereits Implementierungen in den jeweiligen Frameworks.



QA|WARE

Java:

- Resilience4J (Docs: <https://resilience4j.readme.io/docs/getting-started>)

C#:

- Polly (Docs: <https://www.pollydocs.org/>)

Python:

- Tenacity (Docs: <https://tenacity.readthedocs.io/en/latest/>)

Javascript:

- Cackatiel (Docs: <https://github.com/connor4312/cockatiel#readme>)



QA|WARE

Softwarearchitektur testen

Softwarearchitektur kann man testen.



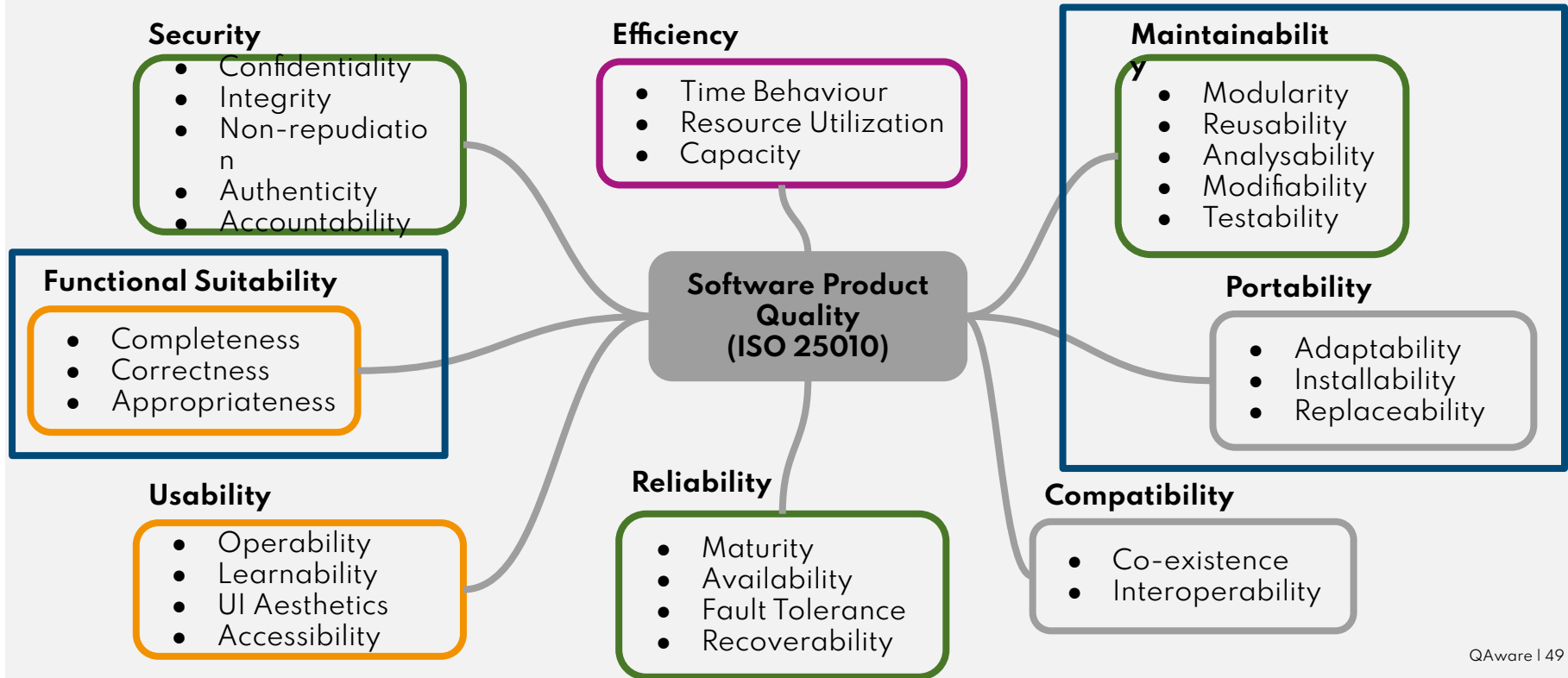
QA|WARE

Welche Aspekte möchte ich bei meiner
Architektur testen?

Der Qualitäts-Oktopus mit seinen “-ilities” hat Einfluss auf die 4 Sichten der Architektur eines Systems.



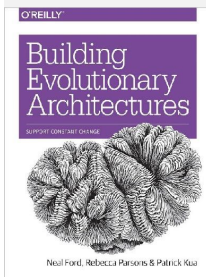
QA|WARE



Architekturen entwickeln sich. Architectural Fitness Functions helfen die “-ilities” kontinuierlich zu validieren.



QA|WARE



TRIAL ?

Borrowed from evolutionary computing, a fitness function is used to summarize how close a given design solution is to achieving the set aims. When defining an evolutionary algorithm, the designer seeks a ‘better’ algorithm; the fitness function defines what ‘better’ means in this context. An **architectural fitness function**, as defined in **Building Evolutionary Architectures**, provides an objective integrity assessment of some architectural characteristics, which may encompass existing verification criteria, such as unit testing, metrics, monitors, and so on. We believe architects can communicate, validate and preserve architectural characteristics in an automated, continual manner, which is the key to building evolutionary architectures.

```
describe "Performance" do
  it "completes a transaction under 2 seconds" do
    expect(k6.check_transaction_round_trip_time()).to < 2
  end
  it "has less than 10% error rate for 10000 transactions" do
    expect(k6.check_error_rate_for_transactions(10000)).to < .1
  end
end

describe "Security" do
  describe "Static Analysis" do
    it "should not have plaintext secrets in codebase" do
      expect(code.has_secrets_in_codebase()).to_not be(true)
    end
  end
  describe "Dynamic Analysis" do
    it "should not have any of the OWASP Top 10" do
      expect(zap.has_owasp_top_10_vulnerabilities()).to be(false)
    end
  end
end
```

Architekturtests sind ein sinnvoller Teil der Testpyramide.



QA|WARE

- Architektur wird in Diagramme gemalt
- Diagramme existieren entkoppelt vom Code
- Verletzungen schleichen sich über die Zeit unbemerkt ein, Diagramme werden nicht aktualisiert
- Architekturverletzungen sind häufig subtil und per Code Review schwer zu sehen

Mit ArchUnit lässt sich Architektur testen.



QA|WARE

“ArchUnit is a free, simple and extensible **library** for checking the **architecture of your Java code** using any plain Java **unit test** framework. That is, ArchUnit can check **dependencies between packages and classes, layers and slices**, check for **cyclic dependencies** and more. It does so by **analyzing given Java bytecode**, importing all classes into a Java code structure. [...]”



ArchUnit verwendet seine eigenen Begrifflichkeiten.



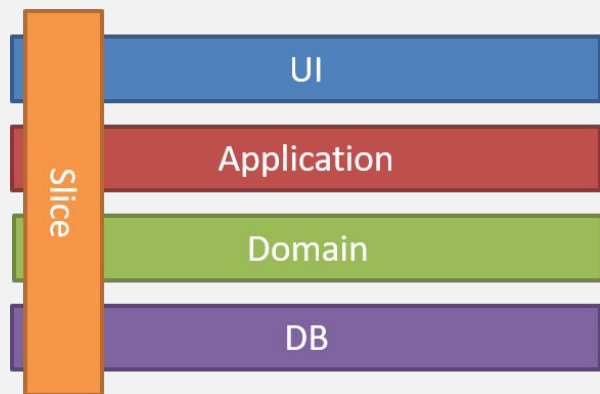
QA|WARE

Slice / Schnitt

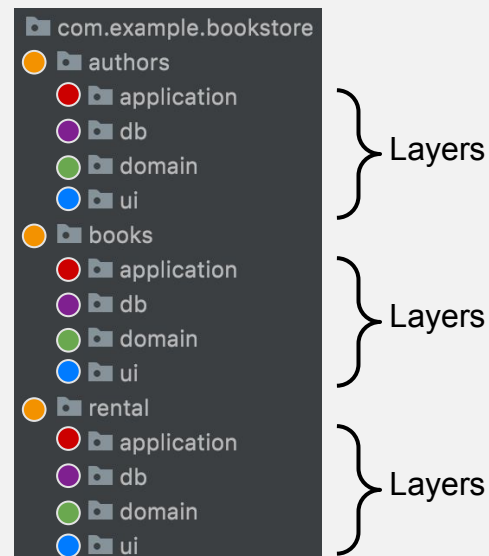
Layer / Schicht

Dependency

Cyclic Dependency



Slices



Wie funktioniert auf mehreren Ebenen.



1. Java Klassen laden
(entweder explizit oder
per Annotation)
2. Architektur-Rule via
Fluent-API definieren
3. Klassen mit Hilfe der
Rule checken

```
// 1

JavaClasses classes = new
ClassFileImporter().importPackages("com.mycompany.myapp");

// 2

ArchRule rule =

    classes().that().resideInAPackage("..service..")

        .should().onlyBeAccessed().byAnyPackage("..controller..",
"..service..");

// 3

rule.check(importedClasses);
```

Was können wir testen? Package Dependencies

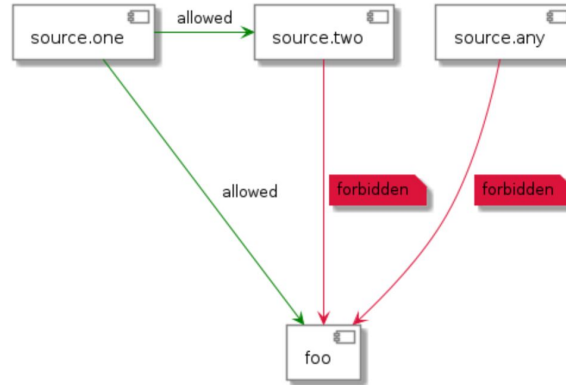


Klasse im Package “source” dürfen nicht auf Klassen im Package “foo” zugreifen.



```
noClasses().that().resideInAPackage("..source..")  
    .should().dependOnClassesThat().resideInAPackage("..foo..")
```

Klassen im Package “foo” dürfen nur aus Klassen in den Packages “source.one” und “foo” verwendet werden.



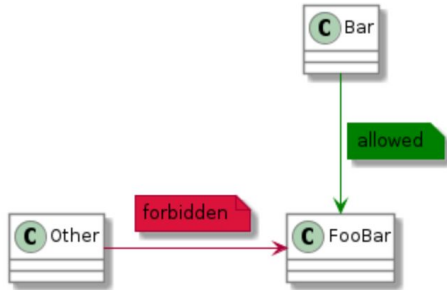
```
classes().that().resideInAPackage("..foo..")  
    .should().onlyHaveDependentClassesThat().resideInAnyPackage("..source.one..", "..foo..")
```

Was können wir testen? Class Dependencies



QA|WARE

Klassen, die auf “Bar” enden, dürfen nur von Klassen, die “Bar” heißen, benutzt werden.



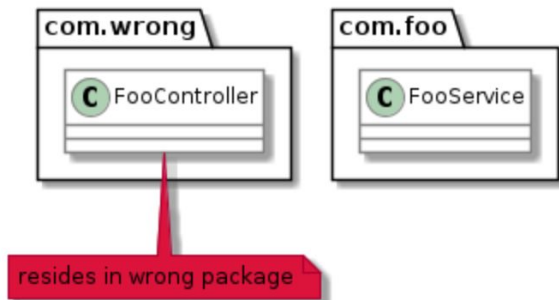
```
classes().that().haveNameMatching(".*Bar")  
    .should().onlyHaveDependentClassesThat().haveSimpleName("Bar")
```

Was können wir testen? Class and Package Containment



QA|WARE

Klassen, die mit “Foo” beginnen, müssen im Package “com.foo” liegen.



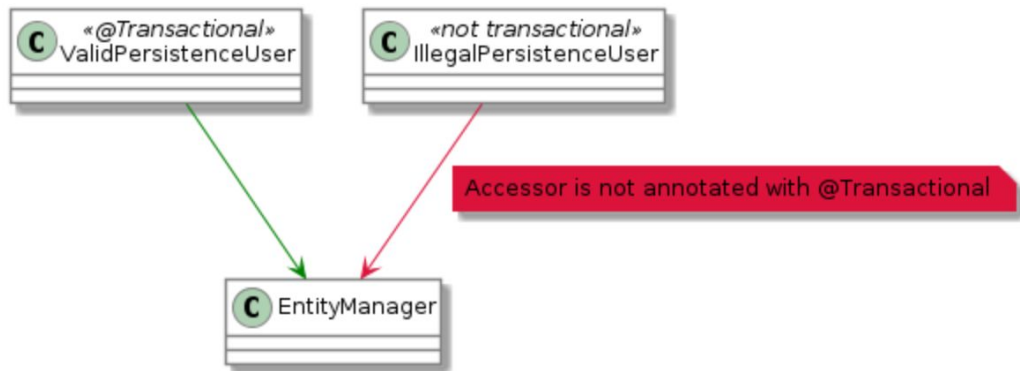
```
classes().that().haveSimpleNameStartingWith("Foo")  
    .should().resideInAPackage("com.foo")
```

Was können wir testen? Annotations



QAWARE

EntityManager (d.h. EntityManager selbst und Sub-Klassen) dürfen nur von Klassen benutzt werden, die mit "Transactional" annotiert sind.



```
classes().that().areAssignableTo(EntityManager.class)
    .should().onlyHaveDependentClassesThat().areAnnotatedWith(Transactional.class)
```

Was können wir testen? Layered Architecture



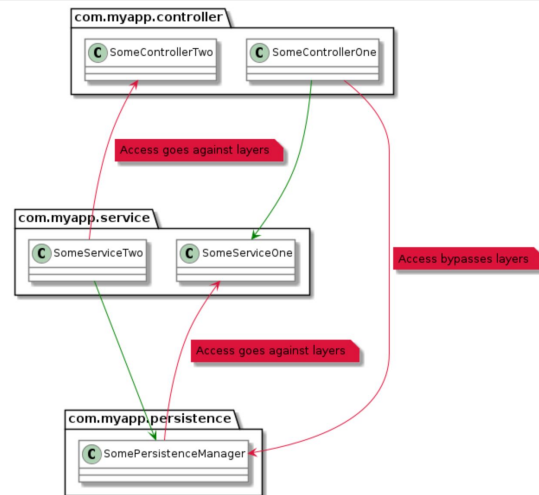
Jede Schicht darf nur auf die gleiche oder die direkt darunter liegende Schicht zugreifen.

Zugriffe “nach oben” sind nicht erlaubt.

Zugriffe, die Schichten überspringen, sind nicht erlaubt.

Die Schichten sind:

- Controller
- Service
- Persistence



```
layeredArchitecture()
    .consideringAllDependencies()
    .layer("Controller").definedBy("..controller..")
    .layer("Service").definedBy("..service..")
    .layer("Persistence").definedBy("..persistence..")

    .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
    .whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
```

Was können wir testen? Cycles

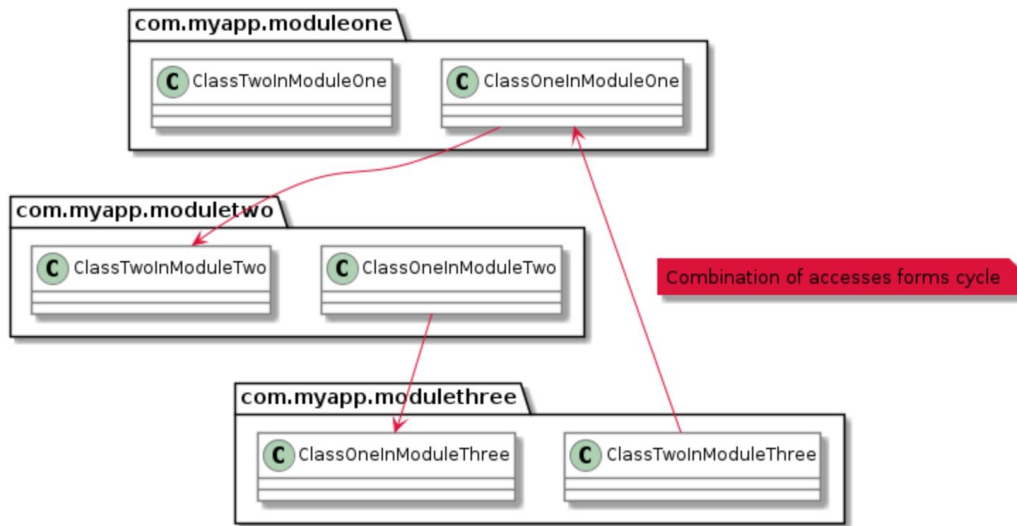


Wir haben eine modulare
Anwendung mit drei Modulen:

- moduleone
- moduletwo
- modulethree

Die Module sind die Packages
unterhalb von “com.myapp”

Die Module der App “myapp”
sollen untereinander keine
zyklischen Abhängigkeiten
haben



```
slices().matching("com.myapp.(*)..").should().beFreeOfCycles()
```

ArchUnit kann auch Diagramme testen.



QA|WARE

- Architekturdiagramme können mit PlantUML (*.puml) beschrieben werden
- Alle Informationen dazu findet ihr unter <https://plantuml.com/>
- Diagramme, die ihr mit Structurizr entworfen habt, könnt ihr direkt als PlantUML exportieren
- Die meisten IDEs bieten Plugins, um PlantUML Diagramme direkt in der IDE gerendert darzustellen

ArchUnit kann auch Diagramme testen.



```
// 1

classes().should(adhereToPlantUmlDiagram(getPlantUmlDiagram(),
consideringOnlyDependenciesInDiagram()))

        .ignoreDependencies(SystemAccessPortImpl.class,
CapeOrganizationHierarchyUtils.class)

// mit .ignoreDependencies lassen sich explizit noch einmal Klassen etc. vom Test

// ausschließen
```